



EL SERVIDOR PERDIDO

EVALUACIÓN FINAL — TALLER DE APLICACIONES PARA INTERNET

 Tecnologías	HTML · CSS · JavaScript · PHP · MySQL / MariaDB
 Modalidad	Individual o grupo

IMPORTANTE

El objetivo es construir una aplicación funcional y personalizada.
Los extractos de código son una guía inicial: cada estudiante debe completarlos, adaptarlos y mejorar la presentación visual con una identidad propia.

1. CONTEXTO DEL CASO

Una empresa tecnológica ha encontrado un antiguo servidor de respaldo en una sala abandonada del datacenter. Para recuperar el acceso a sus archivos, se debe superar una serie de desafíos digitales. Tu misión será construir una aplicación web tipo Escape Room, donde el usuario avance resolviendo pistas. Cada respuesta correcta desbloqueará una nueva pista hasta llegar al final del juego.

2. HISTORIA DEL JUEGO

La aplicación se llamará:

El Servidor Perdido

Al iniciar, el usuario debe ver una historia similar a esta:



TRANSMISIÓN ENTRANTE — AÑO 2035

Un antiguo servidor de respaldo ha sido encontrado en una sala abandonada del datacenter. Para recuperar los archivos, debes superar una serie de pistas digitales.

Cada respuesta correcta desbloqueará el siguiente nivel.
Si fallas, el sistema te dará una advertencia.

¿Podrás recuperar el acceso antes de que el servidor se apague?

3. OBJETIVO DE LA EVALUACIÓN

- Mostrar una historia inicial.
- Cargar pistas desde una base de datos.
- Validar respuestas usando PHP.
- Avanzar progresivamente por niveles.
- Mostrar una pantalla final al completar el escape room.
- Utilizar CSS y JavaScript para mejorar la experiencia del usuario.

4. ESTRUCTURA SUGERIDA DEL PROYECTO

Organiza tu proyecto de la siguiente manera:

```
escape-room/
├── index.php      ← Página de inicio e historia
├── juego.php      ← Lógica principal del juego
├── final.php      ← Pantalla de victoria
├── conexion.php   ← Conexión a la base de datos
├── estilos.css    ← Estilos personalizados (obligatorio modificar)
├── script.js      ← Validación JavaScript
└── database.sql   ← Script SQL de la base de datos
```

5. PARTE 1 — BASE DE DATOS

Crea una base de datos llamada `escape_room` y una tabla llamada `pistas` con la siguiente estructura:

Campo	Tipo sugerido	Descripción
id	INT AUTO_INCREMENT	Identificador único de la pista
pregunta	TEXT	Texto de la pista / enunciado
respuesta	VARCHAR(100)	Respuesta correcta esperada
mensaje_exito	TEXT	Mensaje al responder correctamente
orden	INT	Número de nivel o etapa

Extracto de código de apoyo

```
CREATE DATABASE IF NOT EXISTS escape_room;
USE escape_room;

CREATE TABLE IF NOT EXISTS pistas (
    id            INT AUTO_INCREMENT PRIMARY KEY,
    pregunta      TEXT                NOT NULL,
    respuesta     VARCHAR(100) NOT NULL,
    mensaje_exito TEXT                NOT NULL,
    orden         INT                 NOT NULL
);

INSERT INTO pistas (pregunta, respuesta, mensaje_exito, orden)
VALUES (
    'Soy la estructura principal de una página web. Sin mí, no hay etiquetas ni contenido ordenado. ¿Quién soy?',
    'HTML',
    'Correcto. Has desbloqueado la estructura base del servidor.',
    1
);
```

☒ Debes agregar al menos 3 pistas más para completar un mínimo de 4 niveles.



6. PARTE 2 — CONEXIÓN A LA BASE DE DATOS

Crea el archivo conexion.php. Este archivo permite que PHP se conecte a la base de datos.

```
<?php
$servidor  = "localhost";
$usuario   = "root";
$password  = "";
$base_datos = "escape_room";

$conexion = new mysqli($servidor, $usuario, $password, $base_datos);

if ($conexion->connect_error) {
    die("Error de conexión: " . $conexion->connect_error);
}

$conexion->set_charset("utf8");
?>
```



7. PARTE 3 — PÁGINA DE INICIO

Crea la página index.php con los siguientes elementos:

- Nombre del juego.
- Historia inicial.

- Botón o enlace para comenzar.
- Reinicio del nivel de juego.
- Enlace hacia juego.php.

Extracto de código

```
<?php
session_start();
$_SESSION["nivel"]    = 1;
$_SESSION["intentos"] = 0;
?>

<!DOCTYPE html>
<html lang="es">
<head>
    <meta charset="UTF-8">
    <title>El Servidor Perdido</title>
    <link rel="stylesheet" href="estilos.css">
</head>
<body>
    <div class="contenedor">
        <h1>El Servidor Perdido</h1>

        <p>Año 2035. Un antiguo servidor de respaldo ha sido encontrado
            en una sala abandonada del datacenter.</p>

        <p>Para recuperar los archivos, deberás superar una serie
            de pistas digitales.</p>

        <!-- Agrega aquí un botón o enlace para iniciar el juego -->

    </div>
</body>
</html>
```

8. PARTE 4 — PÁGINA DE JUEGO

Crea juego.php — la página principal del escape room. Debe:

- Iniciar o continuar la sesión.
- Saber en qué nivel está el usuario.
- Consultar la pista actual desde la base de datos.
- Mostrar la pregunta en pantalla.
- Permitir ingresar una respuesta.
- Validar la respuesta con PHP.
- Avanzar al siguiente nivel si la respuesta es correcta.
- Mostrar un mensaje de error si la respuesta es incorrecta.
- Redirigir a final.php cuando se completen todas las pistas.

Extracto — inicio de juego.php

```
<?php
session_start();
include("conexion.php");

if (!isset($_SESSION["nivel"])) { $_SESSION["nivel"] = 1; }
if (!isset($_SESSION["intentos"])) { $_SESSION["intentos"] = 0; }

$nivel = $_SESSION["nivel"];
$mensaje = "";
?>
```

Extracto — consultar pista

```
$sql = "SELECT * FROM pistas WHERE orden = $nivel";
$resultado = $conexion->query($sql);

if ($resultado->num_rows == 0) {
    header("Location: final.php");
    exit();
}

$pista = $resultado->fetch_assoc();
```

Extracto — mostrar la pista

```
<h1>Nivel <?php echo $nivel; ?></h1>

<div class="pista">
    <h2>Pista encontrada</h2>
    <p><?php echo $pista["pregunta"]; ?></p>
</div>
```

Extracto — formulario de respuesta

```
<form method="POST" onsubmit="return validarFormulario();">
    <input
        type="text"
        name="respuesta"
        id="respuesta"
        placeholder="Ingresa tu respuesta"
    >
    <button type="submit">Enviar respuesta</button>
</form>
```

Extracto — validar respuesta

```
if ($_SERVER["REQUEST_METHOD"] == "POST") {

    $respuesta_usuario = trim($_POST["respuesta"]);

    $sql_validar = "SELECT * FROM pistas WHERE orden = $nivel";
    $resultado_val = $conexion->query($sql_validar);
    $pista_validar = $resultado_val->fetch_assoc();

    $_SESSION["intentos"]++;

    if (strcasecmp($respuesta_usuario, $pista_validar["respuesta"]) == 0) {

        // → Aquí debes aumentar el nivel del usuario
        // → Si ya no quedan pistas, redirige a final.php
        // → Si aún quedan pistas, muestra el mensaje de éxito

    } else {
        $mensaje = "Respuesta incorrecta. El servidor rechaza el acceso.";
    }
}
```

9. PARTE 5 — PÁGINA FINAL

Crea final.php — se muestra cuando el usuario completa todas las pistas. Debe incluir:

- Mensaje de felicitación.
- Texto indicando que el servidor fue recuperado.
- Cantidad total de intentos realizados.
- Botón para volver a jugar.

Extracto de código

```
<?php
session_start();
$intentos = isset($_SESSION["intentos"]) ? $_SESSION["intentos"] : 0;
session_destroy();
?>

<!DOCTYPE html>
<html lang="es">
<head>
    <meta charset="UTF-8">
    <title>Acceso Recuperado</title>
    <link rel="stylesheet" href="estilos.css">
</head>
<body>
    <div class="contenedor">
        <h1>¡Acceso recuperado!</h1>

        <p>Has desbloqueado todos los niveles del servidor perdido.
        Los archivos han sido restaurados correctamente.</p>
```

```
<p>Intentos realizados: <?php echo $intentos; ?></p>

<!-- Agrega aquí un botón para volver a jugar -->

</div>
</body>
</html>
```

⚡ 10. PARTE 6 — JAVASCRIPT

Crea script.js. La aplicación debe usar al menos una función JavaScript. El mínimo obligatorio es validar que el campo no esté vacío antes de enviar el formulario.

```
function validarFormulario() {
    let respuesta = document.getElementById("respuesta").value;

    if (respuesta.trim() === "") {
        alert("Debes ingresar una respuesta antes de continuar.");
        return false;
    }

    return true;
}
```

Recuerda enlazar el archivo en juego.php:

```
<script src="script.js"></script>
```

🧐 11. PARTE 7 — PERSONALIZACIÓN VISUAL CON CSS

Además de crear estilos.css, cada estudiante debe personalizar visualmente su Escape Room según la temática elegida.

🚫 **La personalización es OBLIGATORIA.**
No se aceptará el diseño base sin modificaciones significativas.

👤 **Temáticas sugeridas (elige una o inventa la tuya):**

- Hacker / Datacenter abandonado
- Nave espacial
- Laboratorio secreto
- Colegio embrujado
- Agencia de detectives
- Videojuego retro

- Mundo postapocalíptico
- Biblioteca misteriosa
- Inteligencia artificial fuera de control

☑ Requisitos mínimos de personalización CSS (mínimo 6)

El proyecto debe incluir al menos 6 de los siguientes elementos:

- Colores propios relacionados con la temática.
- Fondo personalizado: color, degradado o imagen.
- Botones con estilo propio.
- Bordes, sombras o efectos visuales en los contenedores.
- Tipografía coherente con el ambiente del juego.
- Estilos distintos para mensajes de éxito y error.
- Efecto hover en botones o enlaces.
- Diseño responsivo básico para pantallas pequeñas.
- Uso de clases CSS propias (no solo etiquetas generales).
- Personalización visual de la caja donde aparece la pista.
- Animación simple con CSS.
- Íconos, emojis o elementos decorativos relacionados con la historia.

📄 CSS base (punto de partida — debes modificarlo y mejorarlo)

```
body {
  margin: 0; padding: 0;
  font-family: Arial, Helvetica, sans-serif;
  background: #0d1117;
  color: #e6edf3;
}

.contenedor {
  width: 90%; max-width: 700px;
  margin: 60px auto; padding: 30px;
  background: #161b22;
  border: 1px solid #30363d;
  border-radius: 12px;
  text-align: center;
}

h1 { color: #58a6ff; }

.pista {
  background: #0d1117;
  border: 1px solid #238636;
  border-radius: 10px;
  padding: 20px; margin: 25px 0;
}

input[type="text"] {
  width: 80%; padding: 12px;
  border-radius: 8px; border: none;
  margin-bottom: 15px;
}
```



```
button, .boton {
    display: inline-block; background: #238636;
    color: white; padding: 12px 22px;
    border: none; border-radius: 8px;
    text-decoration: none; cursor: pointer;
}
```

✦ Ejemplos de mejoras esperadas

```
/* Fondo con degradado */
body {
    background: linear-gradient(135deg, #020024, #090979, #00d4ff);
}

/* Botón personalizado */
.boton, button {
    background: #00ff88; color: #0d1117;
    font-weight: bold;
    box-shadow: 0 0 12px #00ff88;
    transition: 0.3s;
}
.boton:hover, button:hover {
    background: #ffffff;
    transform: scale(1.05);
}

/* Caja de pista con efecto visual */
.pista {
    border: 2px solid #00ff88;
    box-shadow: 0 0 20px rgba(0, 255, 136, 0.4);
}

/* Responsividad básica */
@media (max-width: 600px) {
    .contenedor { width: 90%; margin: 30px auto; padding: 20px; }
    input[type="text"] { width: 100%; }
    h1 { font-size: 26px; }
}
```

🧩 12. PISTAS SUGERIDAS

El sistema debe tener mínimo 4 pistas. Puedes usar estas o crear tus propias:

Nivel	Pista
1	Soy la estructura principal de una página web. Sin mí, no hay etiquetas ni contenido ordenado. ¿Quién soy?
2	Soy el lenguaje que da color, forma y estilo a una página web. ¿Quién soy?
3	Trabajo del lado del cliente y puedo validar formularios, mostrar alertas o cambiar elementos de la página. ¿Quién soy?



13. REQUISITOS OBLIGATORIOS

- Tener una página de inicio (index.php).
- Tener una base de datos llamada escape_room.
- Tener una tabla llamada pistas.
- Tener al menos 4 pistas en la base de datos.
- Mostrar las pistas una por una según el nivel actual.
- Validar las respuestas con PHP.
- Avanzar al siguiente nivel si la respuesta es correcta.
- Mostrar mensaje de error si la respuesta es incorrecta.
- Mostrar una pantalla final (final.php).
- Usar CSS propio y personalizado.
- Usar al menos una función JavaScript.



14. RESTRICCIONES

- No se permite entregar solo páginas HTML estáticas.
- No se permite mostrar todas las pistas al mismo tiempo.
- No se permite validar únicamente con JavaScript.
- No se permite escribir las pistas directamente en el HTML.
- No se permite omitir la conexión a base de datos.
- No se aceptará entregar el diseño CSS exactamente igual al base.
- El proyecto debe tener identidad visual propia: colores, fondo, botones y contenedores personalizados.



15. ENTREGABLES

- Carpeta completa del proyecto o ruta github.
- Archivo database.sql.
- Archivos PHP, CSS y JS.
- Aplicación funcionando en navegador local.
- Base de datos cargada con las pistas.



16. PAUTA DE EVALUACIÓN

Criterio	Puntaje
Página de inicio con historia y botón para comenzar	10 pts

Base de datos creada correctamente	10 pts
Tabla pistas con campos adecuados	10 pts
Mínimo 4 pistas registradas en la base de datos	10 pts
Conexión PHP con la base de datos	10 pts
Consulta y visualización de pista actual	10 pts
Validación de respuestas con PHP	15 pts
Avance progresivo entre niveles	10 pts
Pantalla final funcional	5 pts
Uso de JavaScript	5 pts
Personalización visual con CSS	10 pts
Orden y limpieza del código	5 pts
TOTAL	100 pts

Detalle: Personalización Visual CSS

Criterio CSS	Puntaje
Hoja estilos.css correctamente enlazada al proyecto	2 pts
Define una temática visual clara y coherente	2 pts
Personaliza colores, fondo y tipografía	2 pts
Personaliza botones, formularios y contenedores	2 pts
Incluye efectos visuales: hover, sombras o bordes	1 pt
Incluye responsividad básica o animación CSS	1 pt
Total CSS	10 pts